

Secure Digital Banking Platform

Team C

1. Introduction

Secure is a full-stack digital banking platform being built to simulate the core operations of a modern bank. The project is split into a Java Spring Boot backend and an Angular frontend, and is organized into four milestones that add features step by step — from basic account/transfer operations to loans, payments, and identity verification (KYC).

2. Objective

To build a secure, easy-to-use banking application that lets a bank manage:

- Customer accounts and balances
- Money transfers between accounts
- Loan applications and credit checks
- Payments with fraud checks
- Customer identity verification (KYC)

3. Technology Stack

Layer	Technology
Backend	Java 17, Spring Boot 4.0.7 (Spring Web, Spring Data JPA)
Database	PostgreSQL (application) / MySQL (milestone database modules)
Backend Tools	Lombok, Maven
Frontend	Angular 19, Angular Material, Chart.js
Extra Frontend Tool	Tesseract.js (used for document text extraction in KYC)
API Testing	Postman

4. Project Architecture

The project follows a simple client-server architecture:

- The Angular frontend provides the user interface (dashboards, forms, tables).
- It talks to the Spring Boot backend over REST APIs (JSON).
- The backend uses JPA/Hibernate to read and write data in PostgreSQL.

Angular Frontend ---> REST APIs ---> Spring Boot Backend ---> PostgreSQL Database

5. Features by Milestone

The work is organized into four milestones, each adding a new set of banking capabilities.

Milestone 1 — Core Banking

- **Account Creation** – open a new customer account
- **Fund Transfer** – transfer money between two accounts, with checks for invalid accounts and insufficient balance
- **Balance Enquiry** – check the current balance of an account
- **Balance Accuracy Check** – compares the account's stored balance against a balance calculated from all its transactions, to catch mismatches
- **Transaction Dashboard & History** – view past transactions and their details/status

Milestone 2 — Loans & Credit Risk

- **Credit Check** – evaluate a customer's loan eligibility based on their profile and history
- **EMI Calculator** – calculates monthly installment amounts for a loan
- **Loan Origination** – a full loan workflow: pre-qualification → application → underwriting → quality check → funding, with status tracking (e.g., Pending, Approved, Rejected)

Milestone 3 — Payments & Transfers

- **Payment Initiation & Review** – start and confirm a payment
- **Beneficiary Verification** – validate the receiver's account before paying
- **Fraud Check** – screens a transaction for suspicious activity

Milestone 4 — KYC & Identity Verification

- **Document OCR** – extracts text/data from ID documents (using Tesseract.js)
- **Liveness Detection** – confirms a real person (not a photo) during verification
- **Face Match** – compares a selfie with the ID photo
- **Verification Summary** – final KYC result summary

6. Backend Design (Key Points)

Entities: Account, Customer, Transaction are the core database tables, linked using JPA relationships (e.g., one customer can have many accounts).

Layered structure: Each feature (Accounts, Transactions, Credit Check, EMI, Loan Origination, Balance Accuracy) has its own Controller → Service → Repository layers, keeping code organized and easy to maintain.

Error Handling: Custom exceptions like `AccountNotFoundException` are used, along with a `GlobalExceptionHandler` handler, so the API returns clean error messages instead of crashing.

Sample API endpoints:

- `POST /api/transfer/transfer` – transfer funds
- `GET /api/transfer/enquiry/{accountNumber}` – check balance
- `POST /accountCreation` – create a new account

- GET /api/balance-accuracy/{accountNo} – verify balance accuracy
- POST /api/emi/calculate – calculate EMI
- POST /credit-check/evaluate – evaluate loan eligibility
- POST /api/loan-origination – submit a loan application

The screenshots in the repository (using Postman) confirm these APIs were tested manually, including successful transfers, "account not found" errors, and "insufficient balance" errors.

7. Frontend Design (Key Points)

- Built using Angular 19 with a modular, feature-based folder structure (features/balance-accuracy, features/loan-origination, features/payments, features/document-ocr, etc.).
- Uses standalone components and lazy loading, so each page/module loads only when needed — this keeps the app fast.
- Angular Material is used for a clean, consistent look (tables, dialogs, forms).
- Chart.js is used to show analytics (e.g., balance accuracy charts).
- Each milestone has its own dedicated set of pages, all connected through a single routing file (app.routes.ts).

8. Database Design (All Milestones)

The platform's database is organized milestone-by-milestone, with every table linked back to a central customer table. It was built in MySQL (managed via XAMPP and phpMyAdmin) as the shared digital_banking database, with a PHP layer used for direct database connectivity.

Milestone 1 — Core Banking Database

Covers the basic banking data:

- customer – stores customer details
- account – customer accounts and balances
- transactions – deposits, withdrawals, and transfers
- Supports balance accuracy checks and transaction atomicity (so a transfer either fully completes or doesn't happen at all)

PHP scripts handle: database connection, account creation, deposits, withdrawals, balance checks, money transfers, and transaction history.

Milestone 2 — Loan Management Database

Covers the full loan lifecycle using four related tables:

Table	Purpose
loan_application	Stores loan requests (type, amount, tenure, interest rate, status: Pending/Approved/Rejected)
credit_check	Stores credit evaluation for a loan (credit score, income, existing loans, result: Pass/Review/Fail)
emi_calculation	Stores the calculated EMI, total interest, and total payable amount for a loan

Table	Purpose
loan_history	Tracks a loan's status over time (Active/Closed/Defaulted)

Relationships: a customer can have many loan applications; each loan application links to one credit check, one EMI calculation, and its history record.

Loan flow: Application → Credit Check → (Fail → Rejected) / (Pass or Review → Approved) → EMI Calculation → Active Loan → Closed or Defaulted.

EMI formula used:

$$EMI = P \times R \times (1 + R)^N / [(1 + R)^N - 1]$$

where P = principal amount, R = monthly interest rate, N = tenure in months.

Milestone 3 — Payments & Fraud Check Database

Covers beneficiary management, payment processing, and fraud screening using three related tables:

Table	Purpose
beneficiary	Stores payees a customer can pay (name, account no., IFSC, bank name, type: Internal/External, status: Pending/Verified/Blocked)
payment	Stores each payment (sender & receiver account, beneficiary, amount, payment type: Transfer/Bill Payment/Other, mode: IMPS/NEFT/RTGS/UPI, status: Pending/Processing/Success/Failed/Cancelled)
fraud_check	Stores the fraud screening result for a payment (risk score, status: Safe/Suspicious/Blocked, triggered rule, remarks)

Relationships: a customer can have many beneficiaries; a payment links to one sender account, one receiver account, and one beneficiary; each payment has exactly one fraud check.

PHP layer:

- beneficiary_verification.php – validates the customer and target account, then adds a new beneficiary
- payment_initiation.php – checks the sender's balance and beneficiary status before creating a payment
- fraud_check.php – runs a simple rule-based risk check on the payment amount

Fraud risk rule used:

Amount	Risk Score	Fraud Status
≥ ₹1,00,000	80	Suspicious (high amount)
₹50,000 – 99,999	50	Suspicious (medium-high amount)
< ₹50,000	15	Safe

Payment flow: Add & Verify Beneficiary → Initiate Payment (balance & beneficiary checks) → Fraud Check (risk scoring) → Payment Status Updated (Success/Failed/Cancelled).

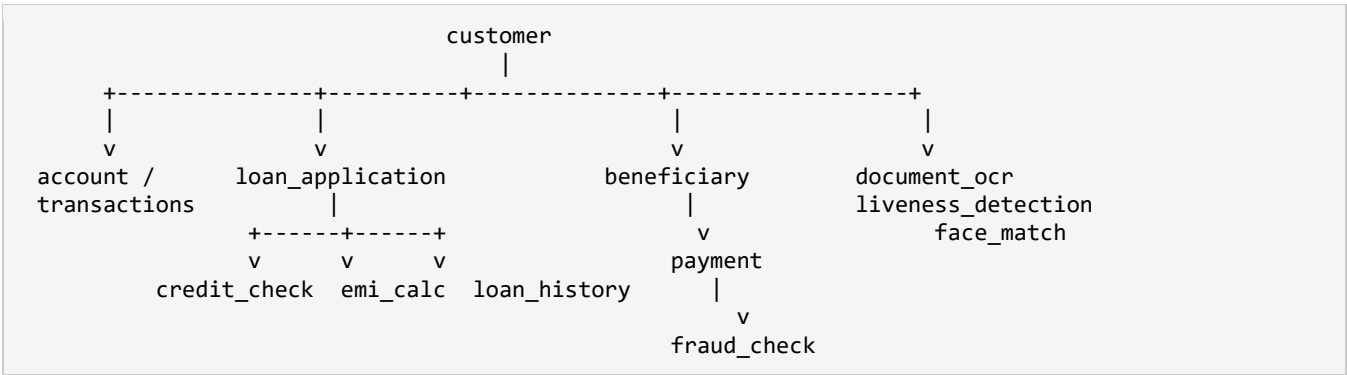
Milestone 4 — AI-Integrated KYC Database

Stores the results produced by the AI verification services, all linked to customer_id:

Table	Purpose
document_ocr	Stores data extracted from ID documents (name, ID number, DOB, address) along with an OCR confidence score and status (Pending/Processing/Verified/Failed)
liveness_detection	Stores the liveness check score and result (Passed/Failed) to confirm a real, live person
face_match	Stores the result of comparing a live photo with the ID photo (SUCCESS/FAILED) with a confidence percentage

KYC flow: Upload Document → AI OCR → Liveness Capture → AI Liveness Check → Face Match → AI Face Match → Overall KYC Verification result.

Overall Database Relationship



Every table uses a primary key (e.g., loan_id, ocr_id, emi_id) and foreign keys back to customer_id or loan_id, so data stays consistent and easy to look up across milestones. NOT NULL, ENUM, and DEFAULT constraints are used throughout to keep the data valid (e.g., loan status always being one of Pending/Approved/Rejected).

9. How the Project Can Be Run

Backend:

1. Set up a PostgreSQL database named Banking.
2. Update the database username/password in application.properties if needed.
3. Run ./mvnw spring-boot:run inside the backend folder.
4. The backend starts on http://localhost:8080.

Frontend:

1. Go to the frontend/balance-accuracy-app folder.
2. Run npm install to install dependencies.
3. Run ng serve (or npm start) to launch the app.
4. Open the app in the browser (usually http://localhost:4200).

10. Conclusion

The Secure Digital Banking Platform (Team C branch) successfully implements the major building blocks of a digital bank — account management, fund transfers, loan processing, payments with fraud checks, and KYC verification — using a clean Spring Boot + Angular architecture. The milestone-based structure makes the project easy to understand, test, and extend with future features.

11. Team-C

Team Member	Role
Manikandan	Frontend Developer
Pavithra	Frontend Developer
Kousalya	Frontend Developer
Nithish	Backend Developer
Jeevana	Backend Developer
Raghvendra	Database Developer